Relatório técnico: Audio Library

EP 2, Técnicas de Programação 2026

`https://github.com/luajaz/tecprog_ep_2`

Luã Jaz Ramos Apostólico, Heitor Macedo Brazil

19 de outubro de 2026

Sumário

1 Problema atacado **2**

2 Arquitetura **2**
2.1 Módulo 1: notas e escalas 2
2.2 Módulo 2: formas de onda 2
2.3 Módulo 3: efeitos 3
2.4 Módulo 4: arquivo WAV 3
2.5 Códigos de retorno 4
2.6 Aplicação principal 4

3 Makefile **5**

4 Testes **5**

5 Versionamento com Git **5**

6 Dificuldades encontradas **6**

7 Resultados obtidos e futuras melhorias **6**

1 Problema atacado

Nosso projeto ataca o problema da geração de áudio a partir de notas musicais providas. Ou seja, é uma aplicação que, dadas notas musicais escolhidas, gera uma onda com perfil escolhido entre senoidal, quadrada, dente de serra e triangular com a frequência das notas escolhidas num buffer, processa opcionalmente a onda com efeitos de ganho, *clipping*, *mixagem*, *delay* e normalização e então escreve o resultado num arquivo de áaudio WAV.

2 Arquitetura

O projeto foi escrito em C e as funções foram distribuídas em quatro módulos:

1. notas e escalas, implementada no arquivo `notes.c`;
2. formas de onda, implementada no arquivo `wave.c`;
3. efeitos, implementada no arquivo `fx.c`;
4. arquivo WAV, implementada no arquivo `wavfile.c`.

2.1 Módulo 1: notas e escalas

O primeiro módulo do projeto, implementado no arquivo `notes.c`, engloba funções que geram notas e escalas musicais, além de funções auxiliares para operar sobre esses objetos. As funções implementadas são:

- `al_midi_to_hz`, que converte uma nota MIDI em frequência (Hz);
- `al_interval_semitones`, que calcula a distância em semitons entre duas notas;
- `al_interval_ratio`, que dá a razão de frequência de um intervalo em semitons;
- `al_scale_generate`, que gera as notas de uma escala (maior, menor, pentatônica, cromática) a partir de uma nota inicial.

Além disso, foi implementado um `struct` para armazenar as informações de uma escala e um macro que relaciona cada escala em uma lista com um nome (`AL_MAJOR`, `AL_MINOR`, `AL_MAJOR_PENTATONIC`, `AL_MINOR_PENTATONIC` e `AL_CHROMATIC`).

2.2 Módulo 2: formas de onda

O segundo módulo do projeto, implementado no arquivo `wave.c`, engloba funções que geram ondas sonoras com certo formato, frequência, amplitude e taxa de amostragem. As funções implementadas são:

- `al_wave_sine`, que gera uma onda senoidal;

- `al_wave_square`, que gera uma onda quadrada;
- `al_wave_saw`, que gera uma onda dente de serra;
- `al_wave_triangle`, que gera uma onda triangular;

Todas as funções recebem um *array* como buffer e seu tamanho, além dos parâmetros da onda.

2.3 Módulo 3: efeitos

O terceiro módulo do projeto, implementado no arquivo `fx.c`, engloba funções que processam um onda com diversos efeitos, além de funções auxiliares para a implementação dos efeitos. As funções principais são:

- `al_fx_gain`, que aplica um efeito de ganho, ou seja, de multiplicação do volume das amostras;
- `al_fx_clip`, que aplica um efeito de *clipping*, ou seja, reduz a amplitude de amostras acima de um certo limite para esse limite;
- `al_fx_mix`, que soma dois buffers;
- `al_fx_delay`, que gera um efeito de eco, duplicando a onda várias vezes com amplitude reduzida e transladada no tempo;
- `al_fx_normalize`, que escala toda a onda para que seu pico tenha uma amplitude específica.

Além disso, as funções auxiliares implementadas foram:

- `al_buffer_peak`, que retorna a maior amplitude de amostra do buffer, ou seja, a amplitude do pico;
- `al_buffer_copy`, que copia as amostras de um buffer para outro;
- `al_buffer_delay`, que translada as amostras de um buffer para o futuro, substituindo o espaço vazio com zeros;
- `al_buffer_advance`, que translada as amostras de um buffer para o passado, substituindo o espaço vazio com zeros.

2.4 Módulo 4: arquivo WAV

O quarto módulo do projeto, implementado no arquivo `wavfile.c`, engloba a função `al_wav_write`, que escreve uma onda contida num buffer em um arquivo WAV, incluindo a conversão de amostras de `float` para `int16_t`, a composição do cabeçalho e a escrita do arquivo em si.

2.5 Códigos de retorno

Foram definidos quatro códigos de retorno diferentes num arquivo `audio_common.h`, sinalizando o estado de saída das funções implementadas:

- (cód. `0`) `AL_SUCCESS`: sucesso
- (cód. `1`) `AL_ERR_INVALID_ARG`: erro de argumento inválido
- (cód. `2`) `AL_ERR_NULL_ARG`: erro de argumento nulo
- (cód. `3`) `AL_ERR_MALLOC_FAIL` : erro na alocação de memória
- (cód. `4`) `AL_ERR_IO`: erro de input/output

2.6 Aplicação principal

A aplicação principal, implementada no arquivo `main.c`, engloba:

- um macro para traduzir códigos de retorno em texto legível;
- a função `parse_note`, que traduz notas escritas com oitava (como "A4", "Cb1", "D#7", etc) em MIDI;
- a função `main`, contendo a interface de uso da aplicação.

Ao ser executada, a função `main` lê as opções dos argumentos providos pela linha de comando e executa diferentes branchs dependendo delas:

- `-w sine|square|saw|triangle`: escolhe o formato de onda;
- `-a amp`: gera uma onda de amplitude "`amp`";
- `-r rate`: gera uma onda com taxa de amostragem "`rate`"em Hz, entre 1 e 384000;
- `-t tempo`: gera uma onda de tempo "`tempo`"em segundos;
- `-o path/to/file`: escreve o arquivo no caminho "`path/to/file`";
- `-g gain`: aplica um efeito de ganho com intensidade "`gain`";
- `-d delay_ms`: aplica o efeito de delay com tempo "`delay_ms`"em milissegundos;
- `-e decay`: aplica o efeito de delay com decay "`decay`", se `-d` também tiver sido passado;
- `-n norm_peak`: aplica o efeito de normalização com valor "`norm_peak`";
- `-c clip`: aplica o efeito de clipping com valor "`clip`".

Dessa maneira, a utilização da aplicação se resume em:

```
audio_library [opções] [nota1] [nota2] [nota3] ...
```

Finalmente, a função sintetiza as notas em um buffer e escreve a header e os dados no arquivo.

3 Makefile

Foi escrita uma Makefile para automatizar o processo de compilação, instalação e execução do projeto. As opções implementadas foram:

- `make lib/libaudio.a`: compila apenas a biblioteca estática;
- `make lib/libaudio.so`: compila apenas a biblioteca dinâmica;
- `make bin/app_static`: compila a aplicação utilizando a biblioteca estática;
- `make lib/app_shared`: compila a aplicação utilizando a biblioteca dinâmica;
- `make test`: executa todos os testes;
- `make run`: compila a aplicação (se necessário) e executa;
- `make install`: instala em `$(PREFIX)` (por padrão, `/usr/local`);
- `make clean`: apaga os diretórios `build/`, `lib/` e `bin/`.
- `make all` (equivalente a apenas `make`): compila todas as opções.

O arquivo Makefile provido permite a compilação da biblioteca de forma dinâmica ou estática. Uma compilação dinâmica permite um arquivo executável menor, mas irá depender mais da instalação local de dependências no sistema operacional em que for executado, enquanto uma compilação estática gera um executável maior, mas é muito mais portável, pois carrega todas as dependências necessárias para a execução.

4 Testes

Foram escritos testes para os módulos 1 e 2 e para funções utilitárias, testando casos normais, assim como casos de fronteira e casos inválidos, permitindo a validação do funcionamento das funções implementadas.

5 Versionamento com Git

Foi usado um repositório simples do GitHub para manter o registro de versões do projeto. Inicialmente usamos um branch para cada módulo, mas depois passamos a utilizar apenas o branch main pois não havia possibilidade de conflito dado o nosso workflow.

O repositório Git conta com um arquivo `README.md` e um `TODO.md` mantendo o briefing do projeto e as tarefas concluídas e pendentes.

Um commit final foi marcado com a tag `v1.0`, demarcando o primeiro *release* final do projeto.

6 Dificuldades encontradas

Dentre as dificuldades encontradas durante o projeto, destacam-se:

- Utilização de arrays muito grandes, com potenciais problemas de overflow;
- Aprendizado do funcionamento de arquivos WAV
- Aprendizado da utilização do Make;
- Questões de comunicação entre membros do grupo e cumprimento de prazos.

7 Resultados obtidos e futuras melhorias

Com esse projeto, obtivemos uma biblioteca de geração de áudio simples funcional, capaz de gerar áudio de forma bastante prática e versátil, além de aplicar diversos efeitos a essas ondas.

Entre as possíveis futuras melhorias do projeto, podemos citar:

- implementação de uma utilização em tempo real, com input MIDI;
- implementação de outros efeitos como reverb, overdrive, chorus, etc;
- implementação de um *piano roll* gráfico, facilitando a composição de melodias;
- implementação de uma biblioteca de acordes, possibilitando a geração automática de acordes de acordo com suas qualidades (maior, menor, suspenso, aumentado, com notas adicionadas como a sétima, nona, etc);
- implementação de um módulo separado de efeitos, permitindo a aplicação em arquivos WAV já escritos previamente.